
Towards Efficient Mixture of Experts with Cache-Aware Routing

Diego Calanzone
Mila, Quebec AI Institute
diego.calanzone@mila.quebec

Riyasat Ohib
Georgia Institute of Technology
PLACEHOLDER@gatech.edu

Pierre-Luc Bacon
Mila, Quebec AI Institute; Université de Montréal
PLACEHOLDER@umontreal.ca

Abstract

Serving very large mixture-of-experts (MoE) language models requires tensor parallelism or offloading expert weights to host memory, both constrained by interconnect bandwidth. Prior work reduces this cost by encouraging the router to reuse a small set of experts per sequence, via architectural changes such as option-critic temporal extension, or custom auxiliary losses with a rank-matching regularizer. We instead treat working-set locality as a reward and optimize it directly with GRPO, using an LRU cache simulator over the router’s own selections as a stateful reward – no custom losses, extra modules, or termination functions. We further combine this reward (CAR-MoE) with causal language modeling on the same rollouts, regularizing distribution shift more effectively than a KL penalty against a frozen reference, and introduce a prompt-conditioning variant letting one policy target different working-set budgets by reading a literal instruction from its own context. We evaluate on downstream task performance, working-set hit ratio, and routing diversity, and find that CAR-MoE avoids the routing collapse that gives an option-critic baseline its near-perfect but degenerate hit ratio.

1 Introduction

Deploying very large mixture-of-experts (MoE) language models, such as DeepSeek-V3 [1] or Kimi K2 [2], requires distributing hundreds of experts across accelerators, either through tensor parallelism or by

offloading expert weights to host memory and streaming them on demand [4]. Even with efficient serving systems such as vLLM [3], throughput degrades sharply once expert transfers become frequent, since moving weights over a comparably slow interconnect starts to dominate compute. This is worsened by the load-balancing objectives used at pretraining time, which explicitly spread router selections uniformly over all experts, at the expense of the locality a working set would need at inference time.

A natural fix is to fine-tune the router to concentrate its choices on a smaller working set per sequence. One line of work draws on the option-critic framework [6] and extends MoE routing temporally, holding an expert selection as a macro-action until a learned termination function switches [5]; option-critic architectures are known to be prone to degenerate solutions and require an explicit deliberation cost to avoid collapsing to one [7], adding training instability on top of the extra termination and policy-over-options modules. MELINOE [8] instead keeps the original routing architecture and adds a cache simulation loss that penalizes expert switching, paired with a rank matching loss against a frozen reference to prevent collapse.

We instead ask whether working-set locality can be optimized directly as a reward with an established policy gradient algorithm, rather than a bespoke loss or architectural change. We simulate a fixed-capacity LRU cache of experts at a single router layer and use its hit ratio, or a soft relaxation, as the reward for GRPO/DAPO [9, 10], whose group-relative advantage and trust region already provide most of the stability a rank-matching term is meant to supply. We further combine this reward with causal language modeling on the model’s own ground-truth completions, reinforcing two trajectories per prompt at once, and find this regularizes distribution shift more effectively than a KL penalty or distillation-style reward. Last, we introduce a prompt-conditioning variant letting one policy target different working-set budgets by reading a literal instruction from its own prompt.

Our contributions are: (i) a simple method, CAR-MoE, that increases MoE inference efficiency using a standard, off-the-shelf policy gradient algorithm rather than a custom loss or architectural change; (ii) a cache-conditioning procedure that trains a single MoE policy to operate at multiple working-set efficiency levels, selectable at deployment time; and (iii) a comparison of existing cache-friendly MoE methods along benchmark performance, efficiency, and routing patterns, revealing an emergent segmentation of experts by contiguous token spans under CAR-MoE.

2 Methodology

2.1 Stateful Working-Set Reward

We simulate a fixed-capacity LRU cache of experts at a single transformer layer ℓ , chosen as the middle layer of the backbone. Let $p_t \in \Delta^E$ be the router’s distribution over E experts at token t , and let K be the number of experts requested per token. We record a request set $e_t = \text{TopK}(p_t, K) \subset \{1, \dots, E\}$ at every position, including the prompt, which is used only to warm the working set and is never rewarded. Let $\mathcal{C}_t \subseteq \{1, \dots, E\}$, $|\mathcal{C}_t| \leq c$, denote the set of experts resident in the working set after token t , for a target capacity c . For each $e \in e_t$, if $e \in \mathcal{C}_{t-}$, the access is a hit and e is promoted to most-recently-used; otherwise it is a miss, e is inserted, and the least-recently-used expert is evicted once the capacity is exceeded. At training time, we route densely at layer ℓ , so that every expert participates in the forward pass weighted by p_t , and use a soft reward, the probability mass already captured by the working set before it is updated,

$$r_t = \sum_{e \in \mathcal{C}_{t-}} p_t[e], \quad R(x, y) = \frac{1}{T} \sum_{t=1}^T r_t, \quad (1)$$

where T is the number of generated tokens and $R(x, y)$ is the sequence-level reward handed to GRPO. This reward is stateful, since the value assigned to a token depends on the entire working-set trajectory up to that point, but it requires no additional trainable parameters and no assumption about which eviction policy the router should be tailored to beyond the choice of c .

2.2 Prompt Conditioning

A single policy trained against a fixed capacity c only ever learns one operating point. We instead let c vary per example and condition the policy on it through its own prompt, so one fine-tuned model deploys at whichever

budget is available. Given a prompt pool $\mathcal{D}$ and candidate sizes $\mathcal{C}$, we construct, for every $x \in \mathcal{D}$ and $c \in \mathcal{C}$, a conditioned prompt $\tilde{x}_c = [\text{CACHE_SIZE}=c] \oplus x$. The augmented set $\mathcal{D}' = \{(\tilde{x}_c, c) : x \in \mathcal{D}, c \in \mathcal{C}\}$ triples the pool and is shuffled so every mini-batch mixes budgets. The target capacity is recovered at reward time by parsing the rollout’s own prompt text, $c = \text{parse}(\tilde{x}_c)$, which makes c a per-example property of a training batch rather than a fixed hyperparameter, so the working set simulated in Equation 1 matches whichever budget the current example asks for.

2.3 Combining Policy Gradients with Supervised Fine-Tuning

GRPO estimates an advantage for a rollout y_g by comparing its reward against the mean and standard deviation of G rollouts sampled from the same prompt, $A_g = (R_g - \text{mean}(R_{1:G}))/\text{std}(R_{1:G})$, reinforcing whichever rollout achieved the highest working-set reward relative to its group. We additionally minimize the teacher-forced negative log-likelihood of the ground truth completion y^* on the same prompt, so the total objective is

$$\mathcal{L}(\theta) = -\mathbb{E}_{x, \{y_g\}} \left[\frac{1}{G} \sum_{g=1}^G A_g \log \pi_{\theta}(y_g | x) \right] + \lambda_{\text{sft}} \text{NLL}(y^* | x), \quad (2)$$

with no KL term against a frozen reference. This reinforces two trajectories per prompt at once – the most working-set-friendly sampled rollout and the one known a priori to be correct – and when they disagree the supervised term keeps the policy anchored to a fluent, correct distribution, which we find regularizes distribution shift more effectively than a KL penalty or distillation-style reward (Appendix A).

2.4 Metrics

At evaluation time we replace the soft reward with its hard counterpart, using deterministic top- K routing to match real deployment behavior. The working-set hit ratio of a rollout is $h_t = \frac{1}{K} \sum_{e \in e_t} \mathbb{1}[e \in \mathcal{C}_{t-}]$, averaged over generated tokens; a working-set miss inserts the requested expert and evicts the least-recently-used one once capacity is exceeded, and we report the mean number of misses per sequence alongside the hit ratio.

3 Experimental Setup

We fine-tune two MoE backbones of different scale, Phi-tiny-MoE-instruct (32 layers, 16 experts, top-2) and OLMoE-1B-7B [11] (16 layers, 64 experts, top-8), using LoRA [12] adapters on the attention and expert projections. Training data is the Nemotron Post-Training Dataset v2 [17], filtered rather than truncated to fit the context budget. The working-set reward is simulated at the middle layer of each backbone, with capacity fixed as a fraction of the expert count and scaled accordingly across backbones. We train with GRPO/DAPO, group size 8, no KL penalty when the supervised term of Section 2.3 is enabled, and compare five objectives per backbone: causal LM alone; the working-set reward alone; their combination (Equation 2); prompt conditioning (Section 2.2); and, as baselines, an option-critic style router [5, 6] and MELINOE [8]. Hyperparameters are in Appendix C.

4 Experimental Results

We report downstream task performance (MMLU [13], MMMLU, GSM8K [14], HumanEval [15], and MATH [16]), working-set efficiency (hit ratio, misses, ECI, throughput), and an interpretability analysis of expert specialization, comparing every objective of Section 3 on both backbones. Results are organized by whether the policy is conditioned on a target working-set size at inference time.

Note on LoRA. Every objective, baselines included, is a LoRA adapter over the frozen backbone, so scores are expected somewhat lower than a full fine-tune, equally for every method. We also do not merge the adapter before timing throughput, so we control for its own inference-time cost by timing the base model’s compute pass plus each objective’s own working-set miss count, rather than its adapter-inflated decode time (Appendix B).

4.1 Without Prompt Conditioning

We refer to the working-set reward objective of Section 2.1 as **CAR-MoE** (Cache-Aware Routing for MoE) from here on, and to its combination with the supervised term of Equation 2 as CAR-MoE (+SFT). Every score below is the mean of two independent runs of the full eval pipeline (same checkpoint, two different sampling seeds for the on-policy generations used by parts 2, 4, and 5 of the pipeline), except OLMoE’s CAR-MoE, option-critic controller, and MELINOE rows, for which only one seed was available at submission time.

Table 1: Results at each backbone’s trained working-set capacity (Phi: $c = 4$, top-2, 16 experts; OLMoE: $c = 16$, top-8, 64 experts). Base row is absolute; other rows are Δ from their own backbone’s base (positive is better). Avg Δ : mean of the five task deltas. Misses: mean working-set misses per sequence (lower better). ECI: normalized routing entropy (1.0 = load-balanced, 0 = collapsed). Our CAR-MoE objectives are highlighted; OLMoE CAR-MoE (+SFT) was still running at submission time (–). See Appendix B for absolute scores and tokens/second.

Objective	MMLU		MMMLU		lm-eval-harness score (Δ from base)				MATH		Avg Δ		working-set efficiency				ECI	
	Phi	OL	Phi	OL	GSM8K	OL	HumanEval	OL	Phi	OL	Phi	OL	Hit ratio	OL	Phi	OL	Phi	OL
Base (untuned)	0.62	0.56	0.34	0.38	0.62	0.68	0.03	0.28	0.09	0.06	–	–	0.57	0.69	649.8	1874.2	0.89	0.90
SFT	+0.00	–0.02	–0.02	–0.01	–0.02	–0.09	+0.01	–0.13	–0.00	–0.02	–0.01	–0.05	0.56	0.68	742.1	1943.9	0.89	0.90
Option-critic	–0.03	–0.04	–0.02	–0.07	–0.11	–0.26	–0.02	–0.14	–0.08	–0.05	–0.05	–0.11	1.00	0.68	0.1	2012.7	0.28	0.89
MELINOE	–0.00	–0.07	–0.01	–0.06	–0.12	–0.31	–0.02	–0.20	–0.07	–0.05	–0.05	–0.14	0.88	0.76	237.7	1541.9	0.67	0.84
CAR-MoE	–0.01	–0.00	+0.00	–0.01	–0.05	–0.01	+0.00	+0.05	–0.01	+0.00	–0.01	+0.01	0.62	0.76	435.9	1004.2	0.89	0.90
CAR-MoE (+SFT)	+0.00	–	+0.00	–	–0.03	–	+0.00	–	–0.01	–	–0.01	–	0.78	–	421.4	–	0.71	–

Phi-tiny-MoE-instruct was pretrained with 16 experts and top-2 routing, while OLMoE-1B-7B was pretrained with 64 experts and top-8 routing, a much larger pool and wider fan-out that matters for the collapse discussed below. On both backbones CAR-MoE is competitive with or better than the base model on most tasks – option-critic and MELINOE instead lose several points of accuracy, most sharply on MATH, GSM8K, and HumanEval, for a comparable or larger hit-ratio gain. CAR-MoE (+SFT) (Phi-tiny-MoE only, so far) raises the hit ratio further still to 0.78 while recovering the reward-only variant’s throughput cost (Appendix B), showing our reward objective combines readily with a supervised term to ground efficiency optimization in supervised learning. On Phi-tiny-MoE, the option-critic controller sharply peaks the routing distribution (ECI falls from 0.89 to 0.28) while reaching a near-perfect hit ratio, which we interpret as routing collapsing onto a handful of experts that stay resident for the entire sequence rather than genuinely tracking locality, at considerable cost to downstream accuracy; CAR-MoE and CAR-MoE (+SFT) drop ECI by far less (to 0.89 and 0.71 respectively). With OLMoE-1B-7B, this collapse does not happen, arguably because of the larger model and its wider top-8 routing: every variant keeps a near-uniform distribution ($\text{ECI} \geq 0.84$), and CAR-MoE is the one that keeps a competitive hit ratio (0.76) together with the benchmark scores closest to the base model’s own.

Figure 1 makes this trade-off explicit across both backbones at once: plotting each objective’s change in average lm-eval-harness score against its working-set hit ratio (both relative to, and at the same capacity as, its own backbone’s base model) separates the two families of method visually. The option-critic controller and MELINOE occupy the upper-left of the plot on Phi-tiny-MoE – the highest hit ratios in the comparison, bought with the largest accuracy regressions – while CAR-MoE and its SFT-combined variant sit closer to the vertical zero-delta line while still improving hit ratio over the base model, and OLMoE’s CAR-MoE is the only point in the entire comparison with a positive score delta together with a hit-ratio gain, falling in the upper-right quadrant we would want every objective to occupy.

4.2 With Prompt Conditioning

The prompt-conditioned policy is trained once per backbone but evaluated at every working-set capacity it was conditioned on, by prefixing each prompt with the target capacity (Section 2.2). Table 4 in Appendix B reports both backbones swept over every capacity they were trained on (Phi-tiny-MoE: $c \in \{2, 4, 8\}$, single seed; OLMoE: $c \in \{8, 16, 32\}$, averaged over two seeds), with capacity expressed as a % of each backbone’s total expert count so the two sweeps share one axis, plotted alongside in Figure 2. As expected, hit ratio im-

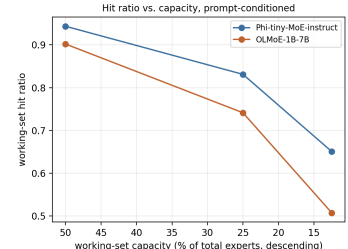


Figure 2: Hit ratio vs. capacity (% of experts), prompt-conditioned policy, both backbones (cf. Table 4).

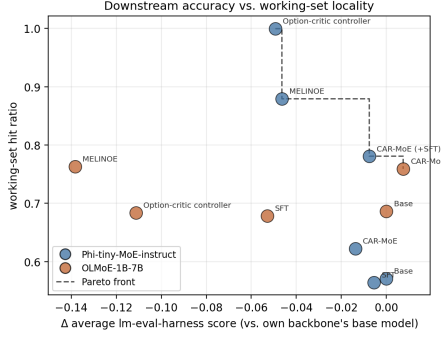


Figure 1: Change in average lm-eval-harness score (mean of MMLU, MMMLU, GSM8K, HumanEval, MATH) versus working-set hit ratio, for every objective on both backbones, relative to each backbone’s own base model. Color distinguishes backbone (Phi-tiny-MoE-instruct vs. OLMoE-1B-7B), each point labeled by objective. The dashed staircase is the Pareto front over both axes jointly (higher hit ratio *and* improved downstream accuracy is better, toward the upper right); only OLMoE’s CAR-MoE combines a positive score delta with a hit-ratio gain, and the option-critic controller sets the frontier’s hit-ratio extreme at the cost of the largest downstream regression.

proves monotonically with capacity on both backbones, from 0.51-0.65 at the smallest capacity to 0.90-0.94 at the largest; tokens/second also increases with capacity, since a larger working set both raises the hit ratio and is evaluated with a fixed generation length, so the throughput gain compounds rather than trading off against the hit-ratio gain as it does for the fixed-capacity CAR-MoE of Table 1. A single prompt-conditioned policy therefore lets a deployment pick a hit-ratio/throughput operating point at inference time without retraining, unlike the fixed-capacity variants of Section 4.1.

For reference, OLMoE’s prompt-conditioned checkpoint evaluated unconditioned on the downstream task suite scores MMLU 0.54, MMMLU 0.37, GSM8K 0.66, HumanEval 0.27, and MATH 0.05 – in the same range as Table 1’s unconditioned objectives, so the conditioning prefix does not measurably harm general task performance. Figure 14 in Appendix D plots the corresponding throughput curves, and the same appendix’s routing traces reveal an *expert chunking* pattern under CAR-MoE – contiguous runs of tokens sharing an expert, resembling the option-critic’s macro-actions but emerging from a stateless per-token reward rather than an architectural hold.

5 Conclusion

We recast working-set-friendly MoE fine-tuning as a policy gradient problem, using an LRU cache simulator as a stateful reward (CAR-MoE) in place of a custom loss or architectural change. Combining it with causal LM on the same rollouts regularizes distribution shift more effectively than a KL penalty, and prompt conditioning lets a single policy serve multiple working-set budgets. On Phi-tiny-MoE, CAR-MoE also avoids the routing collapse that gives the option-critic baseline its near-perfect but degenerate hit ratio, and its routing traces show a "chunking" pattern that emerges without any explicit macro-action machinery.

References

- [1] DeepSeek-AI, Liu, A., Feng, B., Xue, B., et al. DeepSeek-V3 Technical Report. *arXiv preprint arXiv:2412.19437*, 2024.
- [2] Moonshot AI. Kimi K2: Open Agentic Intelligence. *Technical Report*, 2025.
- [3] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient Memory Management for Large Language Model Serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.
- [4] Eliseev, A. and Mazur, D. Fast Inference of Mixture-of-Experts Language Models with Offloading. *arXiv preprint arXiv:2312.17238*, 2023.
- [5] Shen, Y. and Henderson, P. Temporally Extended Mixture-of-Experts via Option-Critic. *arXiv preprint*, 2026.
- [6] Bacon, P.-L., Harb, J., and Precup, D. The Option-Critic Architecture. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2017.
- [7] Harb, J., Bacon, P.-L., Klissarov, M., and Precup, D. When Waiting is not an Option: Learning Options with a Deliberation Cost. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.
- [8] Raje, A., Nayak, A., and Joshi, A. MELINOE: Fine-Tuning Mixture-of-Experts Language Models for Cache-Friendly Offloaded Inference. *arXiv preprint arXiv:2602.11192*, 2026.
- [9] Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., and Guo, D. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*, 2024.
- [10] Yu, Q., Zhang, Z., Zhu, R., Yuan, Y., Zuo, X., Yue, Y., Fan, T., Liu, G., Liu, L., Liu, X., et al. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. *arXiv preprint arXiv:2503.14476*, 2025.
- [11] Muennighoff, N., Soldaini, L., Groeneveld, D., Lo, K., Morrison, J., Min, S., Shi, W., Walsh, P., Tafjord, O., Lambert, N., et al. OLMoE: Open Mixture-of-Experts Language Models. *arXiv preprint arXiv:2409.02060*, 2024.
- [12] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685*, 2021.
- [13] Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring Massive Multitask Language Understanding. *Proceedings of the International Conference on Learning Representations*, 2021.
- [14] Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [15] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*, 2021.
- [16] Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring Mathematical Problem Solving with the MATH Dataset. *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- [17] NVIDIA. Nemotron Post-Training Dataset v2. *Hugging Face Dataset Card*, 2025.
- [18] Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., Chaplot, D. S., de las Casas, D., Hanna, E. B., Bressand, F., et al. Mixtral of Experts. *arXiv preprint arXiv:2401.04088*, 2024.

A Regularizing Distribution Shift Without a KL Penalty

A KL penalty against a frozen reference, or a distillation-style reward that scores a rollout by the reference’s own log-likelihood of it, both anchor the policy to the reference’s original behavior, but neither one ever increases the likelihood the policy assigns to a completion known to be correct, since both are only ever satisfied by staying close to what the reference already does. The supervised term in Equation 2 instead directly minimizes the negative log-likelihood of the ground truth completion under the current policy, which pushes that likelihood up regardless of what the reference would have assigned it, while the working-set reward simultaneously pushes the policy toward whichever sampled rollout was most working-set friendly. In our runs, we find the KL penalty to be redundant once the supervised term is enabled, and disable it accordingly, relying on the supervised term alone to keep the policy from drifting into working-set-friendly but degenerate completions.

B Full Results Tables

Table 1 in Section 4.1 reports every objective’s lm-eval-harness scores as Δ from its own backbone’s base model, to make relative comparisons across five tasks and two backbones easier to read at a glance. Tables 2 and 3 below give the same results in absolute form, together with tokens/second (omitted from the main-text table).

Table 2: Absolute results on Phi-tiny-MoE-instruct (1024 held-out prompts, averaged over two seeds), at the trained working-set capacity ($c = 4$, top-2) – the source values for Table 1’s Phi Δ columns. Tokens/second follows the estimated offloaded-inference latency model (compute plus a per-expert transfer penalty on every working-set miss) rather than a wall-clock trace on real offloading hardware, and is normalized against the base model’s own compute pass (see the LoRA note in Section 4) so that adapters touching the expert projections are not penalized twice.

Objective	MMLU	MMMLU	GSM8K	HumanEval	MATH	Hit ratio	Misses	ECI	Tokens/s
Base (untuned)	0.62	0.34	0.62	0.03	0.09	0.57	649.8	0.89	337.0
SFT	0.62	0.32	0.61	0.04	0.08	0.56	742.1	0.89	370.5
Option-critic	0.59	0.33	0.51	0.01	0.01	1.00	0.1	0.28	438.1
MELINOE	0.61	0.33	0.50	0.01	0.02	0.88	237.7	0.67	440.3
CAR-MoE	0.60	0.35	0.57	0.03	0.07	0.62	435.9	0.89	234.8
CAR-MoE (+SFT)	0.62	0.35	0.59	0.03	0.07	0.78	421.4	0.71	428.6

Table 3: Absolute results on OLMoE-1B-7B (1024 held-out prompts), at the trained working-set capacity ($c = 16$, top-8) – the source values for Table 1’s OLMoE Δ columns. Base and SFT are averaged over two seeds; CAR-MoE, option-critic, and MELINOE are a single seed (see the note in Section 4.1). Tokens/second normalized against the base model’s own compute pass as above. CAR-MoE (+SFT) was still running at submission time and is omitted.

Objective	MMLU	MMMLU	GSM8K	HumanEval	MATH	Hit ratio	Misses	ECI	Tokens/s
Base (untuned)	0.56	0.38	0.68	0.28	0.06	0.69	1874.2	0.90	315.6
SFT	0.55	0.37	0.59	0.15	0.04	0.68	1943.9	0.90	311.1
Option-critic	0.53	0.31	0.42	0.14	0.00	0.68	2012.7	0.89	390.5
MELINOE	0.49	0.32	0.37	0.08	0.00	0.76	1541.9	0.84	438.2
CAR-MoE	0.56	0.37	0.67	0.33	0.06	0.76	1004.2	0.90	273.9

Table 4: Prompt-conditioned policy swept over every working-set capacity it was trained on (1024 held-out prompts per capacity), referenced from Section 4.2. Tokens/second is normalized against each backbone’s own (unconditioned) base model compute pass, as in Table 1. Phi’s row is a single run (no second seed available at submission time); OLMoE’s rows average two.

Capacity (% of experts)	Backbone	Hit ratio	Tokens/s
12.5% (Phi $c = 2$ / OLMoE $c = 8$)	Phi-tiny-MoE	0.65	181.0
	OLMoE	0.51	352.5
25% (Phi $c = 4$ / OLMoE $c = 16$)	Phi-tiny-MoE	0.83	184.4
	OLMoE	0.74	365.6
50% (Phi $c = 8$ / OLMoE $c = 32$)	Phi-tiny-MoE	0.94	186.3
	OLMoE	0.90	378.2

Table 5: Training hyperparameters shared across objectives, per backbone.

	Phi-tiny-MoE-instruct	OLMoE-1B-7B
Experts / top- K	16 / 2	64 / 8
Layers	32	16
Working-set layer	16	8
LoRA rank / alpha	16 / 32	16 / 32
Group size G	8	8
Learning rate	$1e-4$	$1e-4$
Prompt / completion length	1024 / 1024	1024 / 1024

C Hyperparameters

D Expert Routing Traces and Prompt-Conditioned Scaling

Figures 3–14 collect every routing-trace and scaling figure generated for the results of Section 4, for selection into the main text. Each expert-routing-trace figure shows, for 16 held-out prompts (rows) generated on-policy by the given (backbone, objective) pair, the top-1 expert chosen at each of three quartile-spaced layers of the backbone (25%/50%/75% depth) over every generated token (columns), colored by expert id.

Expert chunking. Under CAR-MoE (Figures 5, 11), the router’s per-token expert choice visibly organizes into contiguous runs of tokens assigned to the same expert – segments of consecutive positions sharing an expert id – rather than the token-by-token switching visible in the base model’s trace (Figures 3, 9). We call this pattern *expert chunking*: it resembles the macro-actions an option-critic policy holds explicitly until a learned termination function fires (Figures 7, 12), except here it emerges from a stateless per-token reward rather than being architecturally hard-coded. This suggests the working-set hit ratio itself acts as a training pressure that pushes an ordinary per-token router toward temporally chunked expert use without introducing an explicit option, termination function, or macro-action – the router discovers a soft form of the option-critic’s holding behavior as a side effect of being rewarded for reuse, rather than by construction.

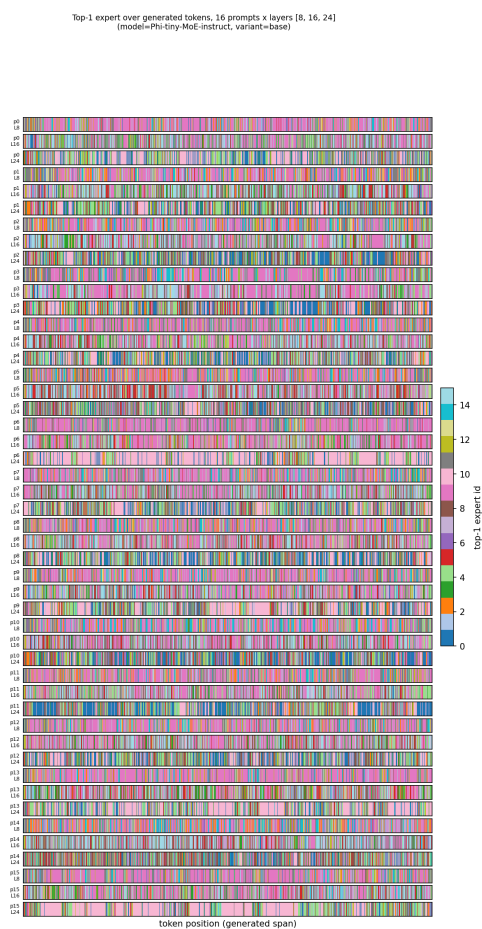


Figure 3: Phi-tiny-MoE-instruct, base (untuned) model – expert routing trace at layers 8/16/24.

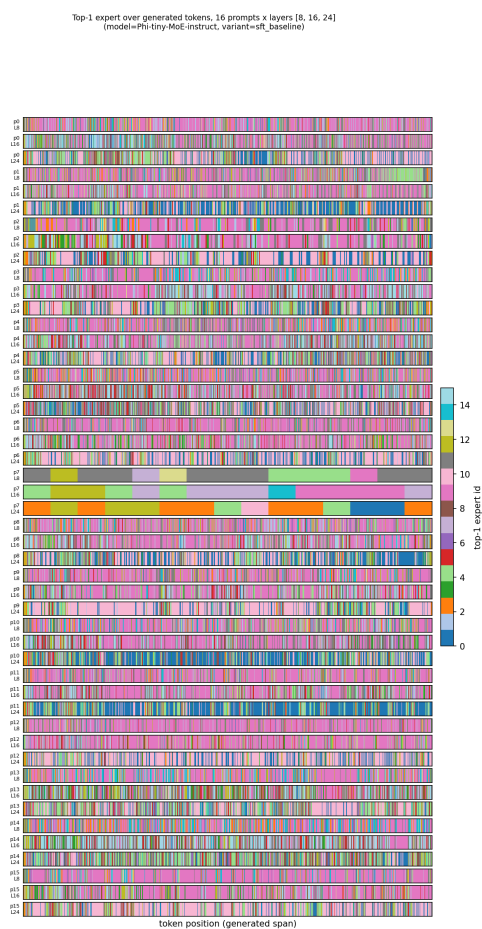


Figure 4: Phi-tiny-MoE-instruct, SFT – expert routing trace at layers 8/16/24.

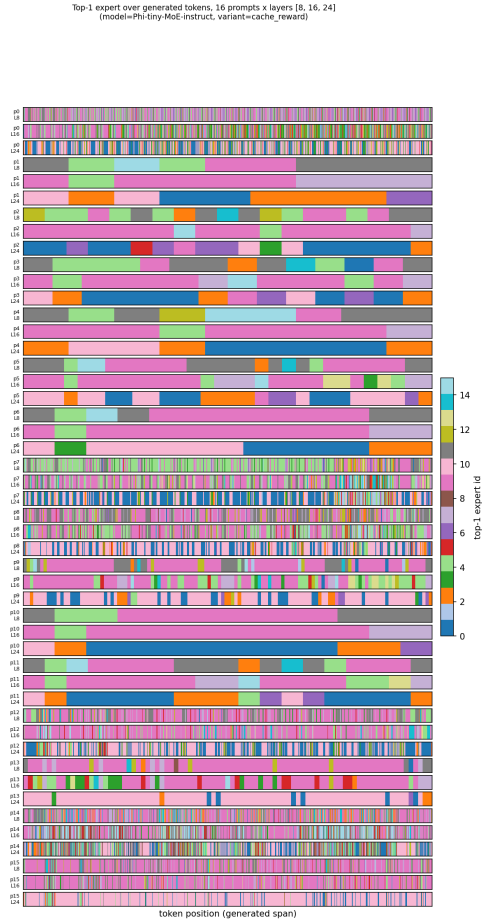


Figure 5: Phi-tiny-MoE-instruct, CAR-MoE – expert routing trace at layers 8/16/24. Note the contiguous runs of tokens sharing the same expert id (expert chunking, discussed above).

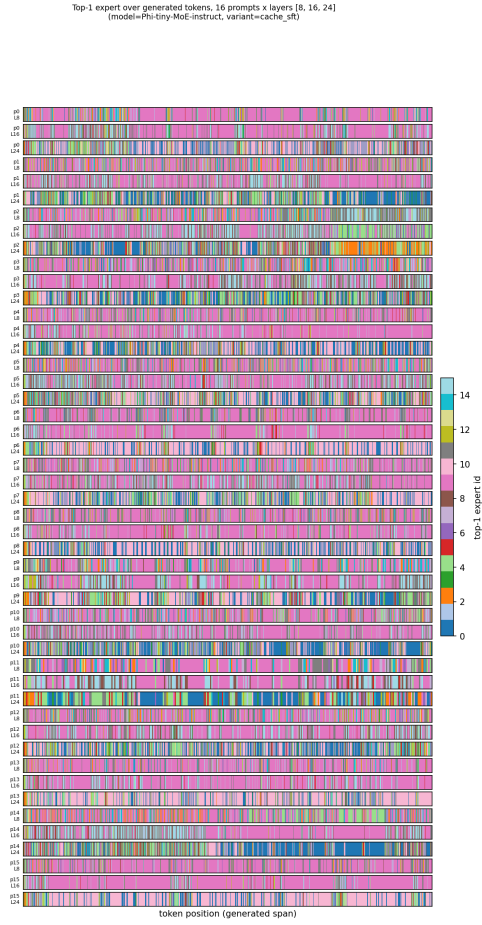


Figure 6: Phi-tiny-MoE-instruct, CAR-MoE (+SFT) – expert routing trace at layers 8/16/24.

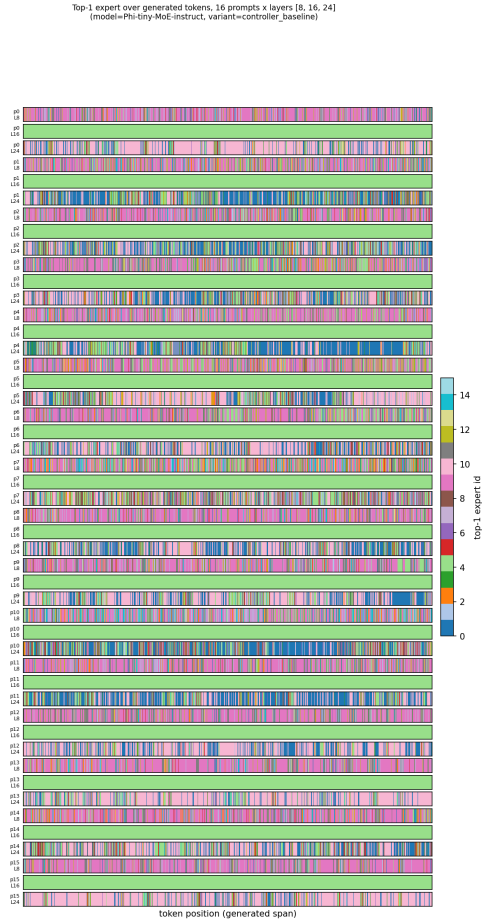


Figure 7: Phi-tiny-MoE-instruct, option-critic controller – expert routing trace at layers 8/16/24. Compare the hard-held macro-actions here against CAR-MoE’s softer chunking in Figure 5.

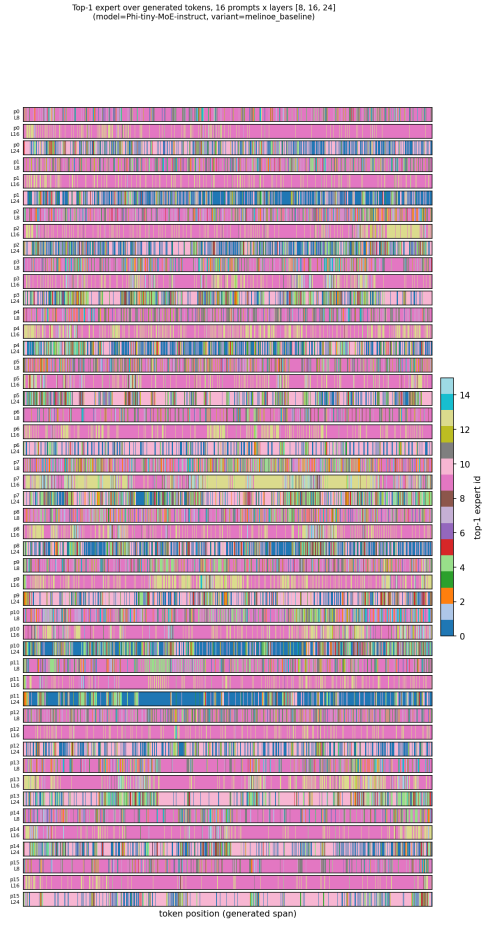


Figure 8: Phi-tiny-MoE-instruct, MELINOE – expert routing trace at layers 8/16/24.

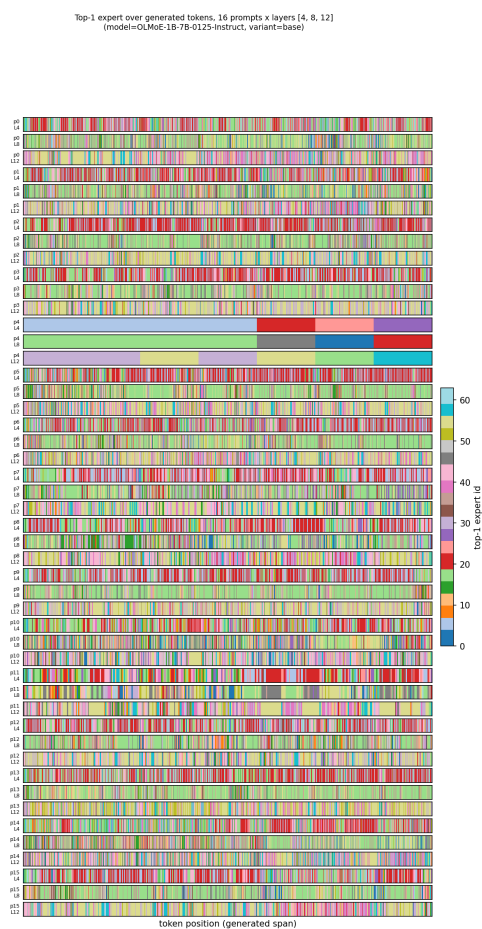


Figure 9: OLMoE-1B-7B, base (untuned) model – expert routing trace at layers 4/8/12.

Top-1 expert over generated tokens, 16 prompts x layers [4, 8, 12]
(model=OLMoE-1B-7B-instruct, variant=sft_baseline)

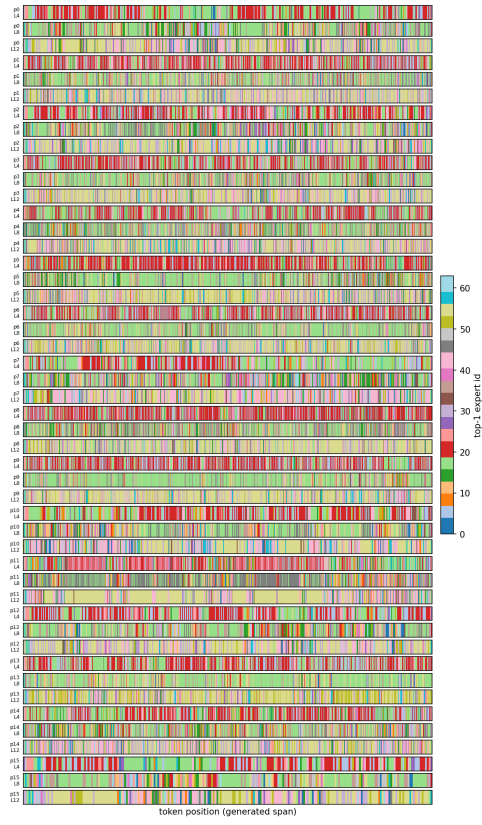


Figure 10: OLMoE-1B-7B, SFT – expert routing trace at layers 4/8/12.

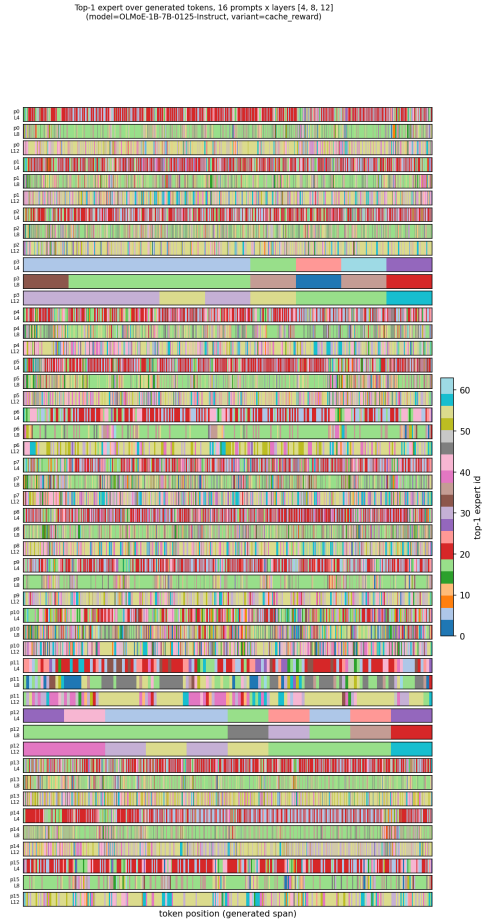


Figure 11: OLMoE-1B-7B, CAR-MoE – expert routing trace at layers 4/8/12. Note the contiguous per-expert segments (expert chunking, discussed above).

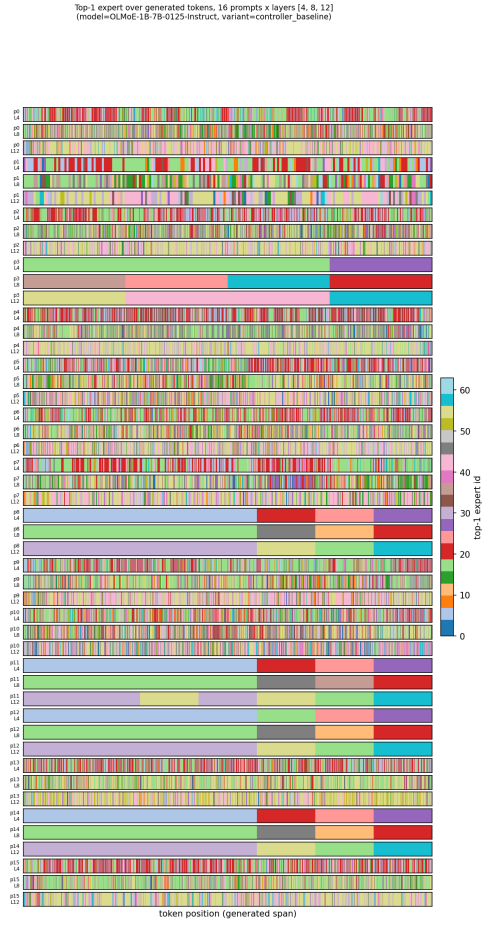


Figure 12: OLMoE-1B-7B, option-critic controller – expert routing trace at layers 4/8/12.

Top-1 expert over generated tokens, 16 prompts x layers [4, 8, 12]
(model=OLMoE-1B-7B-0125-instruct, variant=melinoe_baseline)

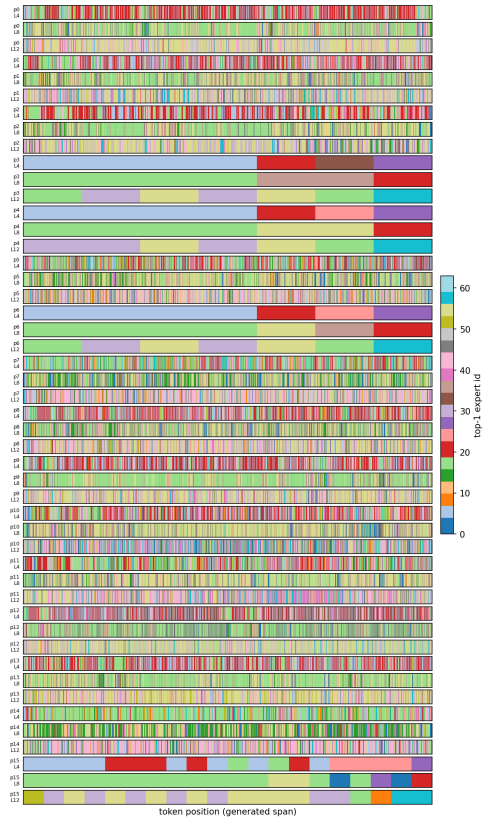


Figure 13: OLMoE-1B-7B, MELINOE – expert routing trace at layers 4/8/12.

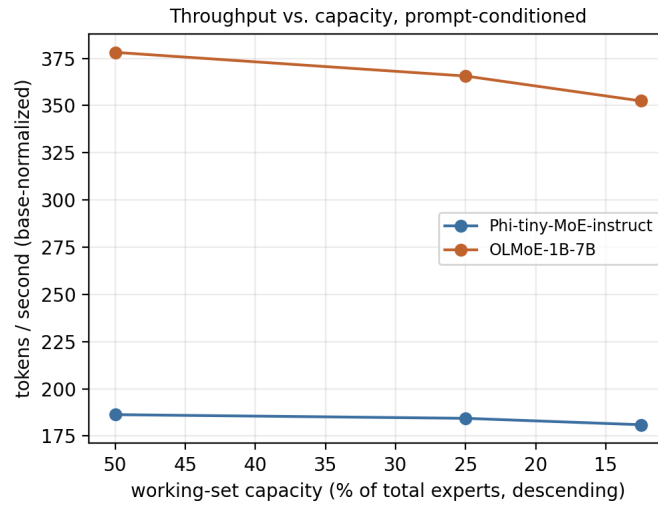


Figure 14: Prompt-conditioned policy, both backbones – base-normalized tokens/second as a function of the conditioned capacity, expressed as % of each backbone’s total expert count (descending on the x -axis), corresponding to Table 4.